Scalable Services (SEZG583)

Scalable Services Group 23

Assignment 2

3 November 2026

SlotHub — Containerization and Deployment Report

Assignment 2: Containerization and Deployment · Part A (Tasks 1–3) + Part B (Minikube, scaling, self-healing)

#	Team member	BITS ID	Main contribution
1	Subhodip Banerjee	2025TM93191	Project lead; Kubernetes, scaling, self-healing
2	J K Shekar	2025TM93095	Compose and troubleshooting (Tasks 2, 3)
3	Ria Mishra	2025TM93010	Docker containerization (Task 1)
4	Harshit Pathak	2025TM93557	—

Links	
Repository	github.com/2025tm93191-stack/slothub
Documents and videos (Drive)	drive.google.com/drive/folders/1daM99AjyiiTwf_5zDP_LFsxPC_0au-pV
Combined team demo	**drive.google.com/file/d/1mZg9wzntKJEhLhzdg5h-_5H-P6iQmdfk/view**
Demo: Subhodip Banerjee	drive.google.com/file/d/1LS7zHjB9VzFTazA1S7hjIFy_Mvd3__MA/view
Demo: J K Shekar	drive.google.com/file/d/1fmjFrybyOXGD_fnKCptb_wWvvc5uQZyv/view
Demo: Ria Mishra	drive.google.com/file/d/1N4OUGaBnn5sqA2gQePTrHN2mgLgliWUp/view

1. Application and deployment architecture

SlotHub (Assignment 1) is a multi-tenant slot-booking platform of four .NET 10 microservices. **Scheduling** owns slots and concurrency-safe holds. **Booking** orchestrates hold → pay → confirm. **Payment** is a mock gateway and **Notification** a mock SMS/email sender. Each service has its own PostgreSQL database and login. The main workflow is create booking (hold a slot), pay, confirm, notify. It exercises synchronous REST calls between services, an outbox, and idempotent retries.

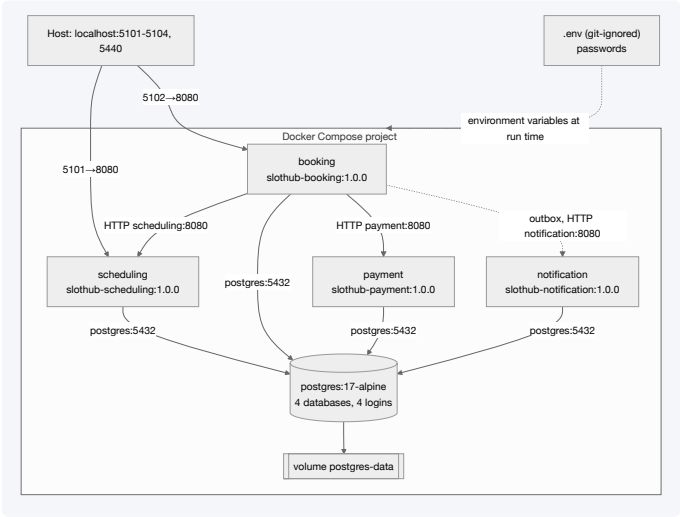


Figure 1: Docker Compose deployment. Containers reach each other by service name on the Compose network; only API ports and the database's host port are published.

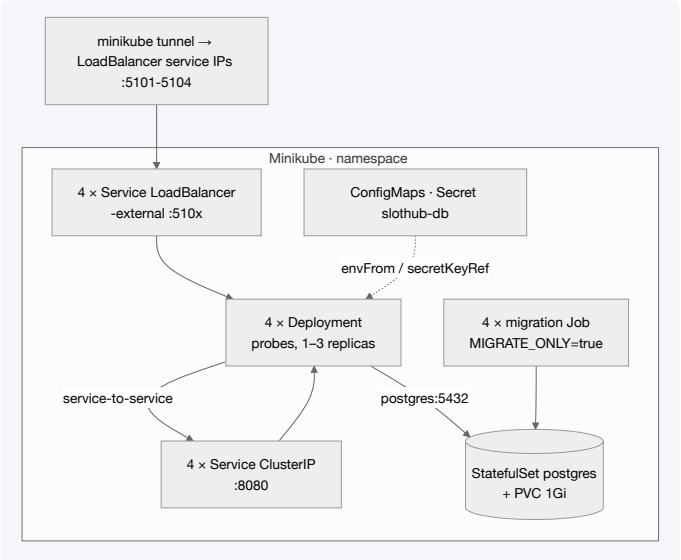


Figure 2: Kubernetes deployment (Part B). Same images, same environment variables; the Secret is created from `.env` by the deploy script.

2. Docker containerization (Task 1)

- **One Dockerfile per service** (`services/<name>/Dockerfile`), multi-stage. The SDK stage restores from project files first (a cached layer) and then publishes. The runtime stage has only the ASP.NET runtime and the app (≈ 386 MB). It

runs as non-root user `1654`, exposes 8080, and carries OCI labels (version, git revision). Images are tagged `slothub-<service>:1.0.0`.
- **Health check in the image:** `HEALTHCHECK` runs `infra/docker/healthcheck.sh`, an HTTP GET of `/health/ready` over bash's `/dev/tcp` (the runtime image has no curl). A container is *healthy* only when the service is up *and* its database is reachable.
- **Configuration only through environment variables:** connection strings, service URLs, timeouts, hold TTL, chaos settings. Nothing environment-specific is in an image.
- **No secrets in images or the repository:** `.dockerignore` excludes `.env`. Passwords come from the git-ignored `.env`, and `docker-compose.yml` uses `${VAR:?…}`, so Compose refuses to start without them. `.env.example` holds `CHANGE_ME` placeholders. The database runs as a container.

Evidence (`evidence/a2-task1-containerization/`, 22/22 checks): `docker compose build --no-cache` with every build stage (raw log attached), 4 separate images with labels, containers created, started and *healthy*; per image, no password in env vars or history and no `.env` file. For the repo, `.env` is untracked and the examples hold placeholders only.

3. Docker Compose deployment (Task 2)

`cp .env.example .env` then **`docker compose up --build`** starts PostgreSQL and the four services. Start-up is **health-gated**: services wait for PostgreSQL to be *healthy*, and Booking waits for Scheduling and Payment to be *healthy*. It only waits for Notification to *start*, because notifications go through an outbox and an SMS outage must never block bookings. Restart policy `unless-stopped` and a named volume keep the data.

Requirement	Evidence (`evidence/a2-task2-compose/`)
All services and the database running	`docker compose ps`: 5 containers *healthy*; `/health/ready` of each service
Communication between microservices	One booking sent with `X-Correlation-Id: trace-…` appears in the logs of all four containers: Booking (held, confirmed), Scheduling (hold reserved, confirmed), Payment (charge approved, idempotency key stored), Notification (2 messages queued via the outbox). We made the outbox forward the correlation ID for this, with a new migration.
Main workflow	Full `e2e-booking.sh`, 26 checks: hold → pay → confirm → SMS + email; declined payment releases the slot; gateway outage → retries, circuit breaker, then recovery with exactly one charge
Logs and health	`docker compose logs` excerpt per service; `docker inspect` health state and check configuration
Stop and restart	9/9: `stop` → `start`, `down` → `up` (containers and network removed, volume kept), `restart booking`. A booking confirmed before each step is still there and new bookings succeed

4. Situated learning: deployment troubleshooting (Task 3)

Problem. In Assignment 1, the first `docker compose up` failed before any service started: `ports are not available: exposing port TCP 0.0.0.0:5432 … bind: address already in use`. Every container stayed `Created`. Moving the mapping to 5433 failed the same way. The code had worked in development, where `dotnet run` used a PostgreSQL installed on the laptop.

Diagnosis. `scripts/a2-troubleshoot-demo.sh` reproduces it safely: a throw-away process holds the database's host port. Then we used:

Command	Finding
`docker compose ps -a`	all containers `Created`: the failure is at container start, not inside an application
`docker inspect slothub-postgres` (`.State`, `.HostConfig.PortBindings`)	state `created`; it is trying to bind container 5432 to a host port
`docker compose logs postgres`	0 lines: PostgreSQL never ran, so the cause is outside the container
`lsof -nP -iTCP:<port> -sTCP:LISTEN`	a non-Docker process owns the port (originally the laptop's own PostgreSQL; 5433/5434 belonged to other projects)

Solution and validation. The host side of the mapping was hard-coded. It became configuration with an unused default, `"${POSTGRES_PORT:-5440}:5432"`, set in `.env`. Container-to-container connections keep using `postgres:5432`, and no code or image changed. After the fix, all 5 containers are healthy, a booking is held, paid and confirmed, and the database answers on the new host port (12/12 checks; full workflow in Task 2 evidence).

Learning. Host ports are shared and scarce; container ports are not. Publish as few as possible, make them configurable, and never publish the database in production. "Works on my machine" hid a dependency on a locally installed database, and containerization made the environment explicit. Diagnose from the outside in (state → configuration → logs → host), where empty logs pointed away from the application. Platform details matter: on macOS, Docker publishes ports on `[::]`, so only a dual-stack listener collides. In a real deployment we would run pre-flight port checks, keep ports in environment variables, use health-gated start-up, and in Kubernetes use ClusterIP Services so internal components need no host ports at all.

5. Advanced: Minikube deployment, scaling and self-healing (Part B)

- **Manifests** (`deploy/k8s/`): namespace; PostgreSQL StatefulSet with a 1 Gi PVC and an init script from a ConfigMap (4 databases, 4 logins); a ConfigMap per service; 4 migration **Jobs**; per service a Deployment (startup and liveness probes on `/health`, readiness on `/health/ready`, requests and limits), a ClusterIP Service for in-cluster calls and a LoadBalancer Service on 5101–5104 for the host (`minikube tunnel`).
- **Secret** `slothub-db` is created from `.env` by `scripts/k8s-deploy.sh` and is never committed. Pods build the connection string with `$(DB_PASSWORD)`.
- **Migrations once, not per replica:** the same image with `MIGRATE_ONLY=true` migrates and exits (new code path), so replicas never migrate concurrently.
- **Images** are built with Docker and streamed into the node's containerd (`docker save | ctr images import`), with no registry.

Demonstration	Result (`evidence/a2-advanced-k8s/`)
Main workflow through Kubernetes	17/17: objects healthy, Jobs completed; one booking traced through the Booking, Scheduling, Payment and Notification **pods**; full `e2e-booking.sh` 26/26 and Challenge 1 (1 of 20) **through the Kubernetes Services**
Scaling	`kubectl scale … --replicas=3` (Scheduling, Booking): 30 simultaneous holds through the load-balanced Service were spread over all 3 pods, with still exactly 1 success. With 3 hold-expiry workers, each of 5 expired holds was released exactly once (`FOR UPDATE SKIP LOCKED`)
Self-healing	A Booking pod was deleted while 6 bookings were made: all 6 confirmed, a replacement pod was created and 3/3 were ready again. The only Payment pod was deleted, recreated, and the flow worked
Readiness	PostgreSQL scaled to 0: all service pods `0/1` and their endpoints `ready=false`. Back to 1: pods ready, data intact on the PVC

> **Why scaling is safe:** no state lives in a pod. Capacity is one atomic SQL update, idempotency a database unique key, and expiry uses `SKIP LOCKED`, the Assignment 1 challenge mechanisms. **Kubernetes problem we hit:** the first deploy imported images built before the `MIGRATE_ONLY` change, so the migration Jobs ran a web server and never completed (`kubectl get pods` showed them `Running`, `kubectl logs` showed `relation "holds" does not exist`). The fix was to rebuild and re-import. Lesson: give each build a distinct tag.

6. Validation, design decisions and future work

The Part A evidence shows image builds, health-gated Compose startup, a traced four-service booking, persistent bookings after restart, and a diagnosed and resolved PostgreSQL host-port conflict. The Part B evidence shows successful migration Jobs, pod readiness, multi-replica concurrency safety and replacement of failed pods. The raw checks and screenshots remain under `evidence/a2-task1-containerization/`, `evidence/a2-task2-compose/`, `evidence/a2-task3-troubleshooting/` and `evidence/a2-advanced-k8s/`.

Decision / trade-off	Reason and limitation
Docker Compose for local deployment	Reproducible five-container setup with health checks and named-volume persistence; not a multi-node production orchestrator.
Separate databases, one PostgreSQL instance	Preserves service-level data ownership with limited local resource use; the database instance is still a shared failure point.
Kubernetes Deployments + Jobs + StatefulSet	Separates application scaling from one-time migrations and persistent database storage; local Minikube does not provide production-grade high availability.
Future improvements	Unique immutable image tags, automated deployment checks in CI, production Secret management, metrics/tracing, and database backup and recovery validation.

Appendix A: Individual contribution summary

Member	Contribution to Assignment 2	Share
Subhodip Banerjee (2025TM93191) Repository owner and project lead	Initiated and guided the project, coordinated architecture, implementation, integration and team reviews. Led the Kubernetes deployment and advanced-task presentation: Minikube, application Deployments and Services, one-time migration Jobs, database persistence, horizontal scaling and pod self-healing. Coordinated the final technical walkthrough and conclusion.	**33⅓%**
J K Shekar (2025TM93095)	Completed and recorded the Docker Compose deployment and troubleshooting demonstrations (Tasks 2 and 3): healthy-service readiness, cross-service booking verification, correlation-ID traces, restart and data-persistence checks, PostgreSQL host-port conflict diagnosis, configuration fix and recovery. Collected deployment evidence, exercised Kubernetes verification, and corrected the Kubernetes trace-check script.	**33⅓%**
Ria Mishra (2025TM93010)	Contributed to the Payment Service container packaging and integration work. Prepared and presented Docker containerization (Task 1): multi-stage service Dockerfiles, build and runtime images, non-root execution, readiness health checks, configuration via environment variables, image inspection and healthy-container verification.	**33⅓%**
Harshit Pathak (2025TM93557)	No individually attributable Assignment 2 implementation or evidence contribution was established in the materials reviewed; no unverified work is assigned.	**0%**

Contribution basis. The three active members share the reported Assignment 2 contribution equally (one-third each), based on the agreed division of implementation support, demonstration preparation, verification and recorded presentation responsibilities. Subhodip's additional role as repository owner, original initiator and technical coordinator is recognized in his contribution description. This is a team-proposed allocation, not a formula calculated from Git commit counts; confirm it reflects actual work and approve it before submission. The individual video segments demonstrate the responsibilities assigned above.

Appendix B: Contribution and learning reflections

These are draft contribution summaries; each member should confirm the wording reflects their actual experience before submission.

Subhodip Banerjee. As repository owner and project lead, coordinated the design and deployment approach and presented the advanced Kubernetes component. The main learning was how to separate persistent database state and one-time migrations from disposable application replicas while preserving correctness during scaling and pod replacement. A future improvement is immutable container image tags and automated deployment checks in CI.

J K Shekar. Demonstrated Docker Compose verification, restart persistence and live port-conflict troubleshooting, including the configurable PostgreSQL host port and service-to-service networking. Also exercised the Kubernetes verification and scaling checks and improved the trace-log test logic. Important lessons were distinguishing host networking from container networking and making automated smoke tests robust. A future improvement is portable network detection for verification scripts.

Ria Mishra. Contributed Payment containerization work and demonstrated the four service images, Dockerfiles, environment-based configuration and container health checks. The key learning was how reproducible, non-root multi-stage images support deployment consistency while keeping secrets out of source control and image layers. A future improvement is broader automated image security and packaging validation in CI.

Harshit Pathak. No verified Assignment 2 changes or personal reflection were available in the materials reviewed. A personal reflection should come from the member rather than be invented.

Appendix C: AI-use declaration

Tool	Used for
Claude (Anthropic), via Claude Code	Turning the brief into a backlog; writing the Dockerfile changes, health-check script, Compose changes, Kubernetes manifests, deploy/verification/troubleshooting scripts and the first draft of this report; diagnosing failures found while running them.
ChatGPT (OpenAI)	Reviewing terminal outputs, diagnosing Compose database authentication and Minikube networking issues, identifying verification-script bugs, planning evidence capture, and drafting/revising demonstration narration and report text. Suggestions were checked against actual execution results.

AI-assisted code, documentation and troubleshooting suggestions were evaluated through local builds, API calls, automated checks and saved evidence. The team retains responsibility for technical correctness and must confirm the declaration reflects each member's actual AI-tool use before submission.

Appendix D: How to reproduce

```
cp .env.example .env                    # set passwords
docker compose up --build               # Part A deployment
./scripts/a2-build-evidence.sh --no-cache && ./scripts/a2-verify-deployment.sh && ./scripts/a2-restart-
demo.sh
./scripts/a2-troubleshoot-demo.sh       # Task 3
docker compose stop && ./scripts/k8s-deploy.sh --skip-build  # use images already built by Compose
minikube tunnel                         # run in a second terminal
kubectl -n slothub get pods,svc,jobs,pvc # check LoadBalancer EXTERNAL-IP values
# export SCHEDULING_URL, BOOKING_URL, PAYMENT_URL, NOTIFICATION_URL from EXTERNAL-IP values
./scripts/a2-k8s-verify.sh && ./scripts/a2-k8s-scale-heal.sh
```